

LAPORAN PROGRES PROYEK

MTA Subway Hourly Ridership

Minggu 3: Data Wrangling – *Gathering Data*

Nama kelompok	Kelompok Smile
Anggota	Muhammad Abil Khoiri (101032330094); Firdaus Arif Ramadhani (101032300131)
Topik proyek	Analisis dan Prediksi Volume Kepadatan Penumpang Kereta Bawah Tanah (<i>MTA Subway Hourly Ridership</i>) Menggunakan Integrasi Data <i>Real-Time</i> GTFS
Notebook Google Colab	https://colab.research.google.com/drive/1GU64Yx-8Bq7PgxiG-lg5t9dnfb1IKIH5
Repositori GitHub	https://github.com/fxrdhan/mta-subway-bigdata
Dataset Final	https://drive.google.com/file/d/1NqbE12P5M6n3l-Q3FYXfoErJgZijpI3k/view?usp=sharing

I. Dataset

Dataset kami adalah **MTA Subway Hourly Ridership: Beginning 2025** dari portal NY Open Data. Dataset difiksasi sebagai *snapshot* dengan batas waktu (*cutoff*) 16 September 2026 pukul 23:00, yaitu data terbaru yang tersedia saat pengambilan. Karena isi server terus berubah setiap pembaruan mingguan, hasil pengambilan disimpan sebagai file Parquet final; file inilah dataset yang difiksasi.

Tabel 1. Identitas dataset

Item	Keterangan
Nama dataset	MTA Subway Hourly Ridership: Beginning 2025
ID dataset	5wq4-mkjj
Pemilik	<i>Metropolitan Transportation Authority</i> (MTA)
Sumber	https://data.ny.gov/d/5wq4-mkjj
Akses data	SODA API https://data.ny.gov/resource/5wq4-mkjj.csv
Periode <i>snapshot</i>	1 Januari 2025 00:00 sampai 16 September 2026 23:00 (624 hari)
Jumlah data	45.243.727 baris
Jumlah fitur	12 kolom
Target/label	Tidak ada label bawaan; target proyek adalah <i>ridership</i>

Alasan pemilihan dataset adalah sebagai berikut.

- Dataset ini sesuai dengan topik proyek. Jumlah penumpang tercatat per jam untuk setiap kompleks stasiun, sehingga dapat dipakai untuk menganalisis dan memprediksi kepadatan penumpang per jam.
- Merupakan data resmi dari operator (MTA) dan diperbarui setiap minggu.

- Memuat metode pembayaran (OMNY dan MetroCard) yang dibutuhkan untuk analisis adopsi pembayaran nirsentuh.
- Dapat diambil langsung lewat API tanpa autentikasi, sehingga proses pengumpulan dapat diulang dan diverifikasi.
- Memiliki ID dan koordinat stasiun, sehingga dapat dihubungkan dengan data *streaming* GTFS-RT pada tahap berikutnya.

Satu baris dataset mewakili jam $\times$ kompleks stasiun $\times$ metode pembayaran $\times$ kelas tarif. Dataset tidak memiliki label bawaan. Kolom *ridership* menjadi target regresi (jumlah penumpang per jam), sedangkan label jam sibuk (*peak/off-peak*) untuk klasifikasi akan diturunkan dari kolom ini pada tahap berikutnya.

Tabel 2. Kolom dataset

No	Kolom	Tipe	Keterangan
1	<i>transit_timestamp</i>	<i>timestamp</i>	Waktu lokal New York, dibulatkan ke bawah ke jam
2	<i>transit_mode</i>	teks	subway, staten_island_railway, atau tram
3	<i>station_complex_id</i>	teks	ID kompleks stasiun
4	<i>station_complex</i>	teks	Nama kompleks stasiun beserta rutenya
5	<i>borough</i>	teks	Wilayah administratif
6	<i>payment_method</i>	teks	omny atau metrocards
7	<i>fare_class_category</i>	teks	Kelas tarif
8	<i>ridership</i>	numerik	Jumlah penumpang yang masuk (target)
9	<i>transfers</i>	numerik	Bagian <i>ridership</i> yang masuk lewat transfer gratis
10	<i>latitude</i>	numerik	Lintang stasiun
11	<i>longitude</i>	numerik	Bujur stasiun
12	<i>georeference</i>	teks	Titik lokasi format WKT POINT (bujur lintang)

Dataset 2020–2024 memiliki skema yang sama, tetapi tidak dimasukkan karena ruang lingkup proyek difiksasi pada data 2025 dan seterusnya. Sumber *streaming* (GTFS-RT) dikumpulkan pada tahap terpisah.

II. Sumber dan Cara Memperoleh Data

Data diambil langsung dari sumber utama melalui SODA API portal NY Open Data, menggunakan permintaan HTTP GET (pustaka *requests*) di Google Colab. Tiga endpoint dipakai:

- `/api/views/5wq4-mkjj.json` untuk membaca metadata dan memastikan sumbernya benar (Gambar 1).

- `/resource/5wq4-mkjj.json` untuk melihat bentuk mentah respons. Pada format ini seluruh angka dikirim sebagai teks dan georeference berupa objek GeoJSON (Gambar 2).
- `/resource/5wq4-mkjj.csv` untuk pengambilan data, karena lebih ringkas dan dapat dibaca langsung dengan tipe kolom yang ditetapkan.

Kueri agregat di sisi server (`$select`, `$where`, `$group`) juga dipakai untuk memfiksasi *snapshot* dan sebagai acuan validasi.

Metadata diambil dari endpoint `/api/views` untuk memastikan sumbernya benar.

```

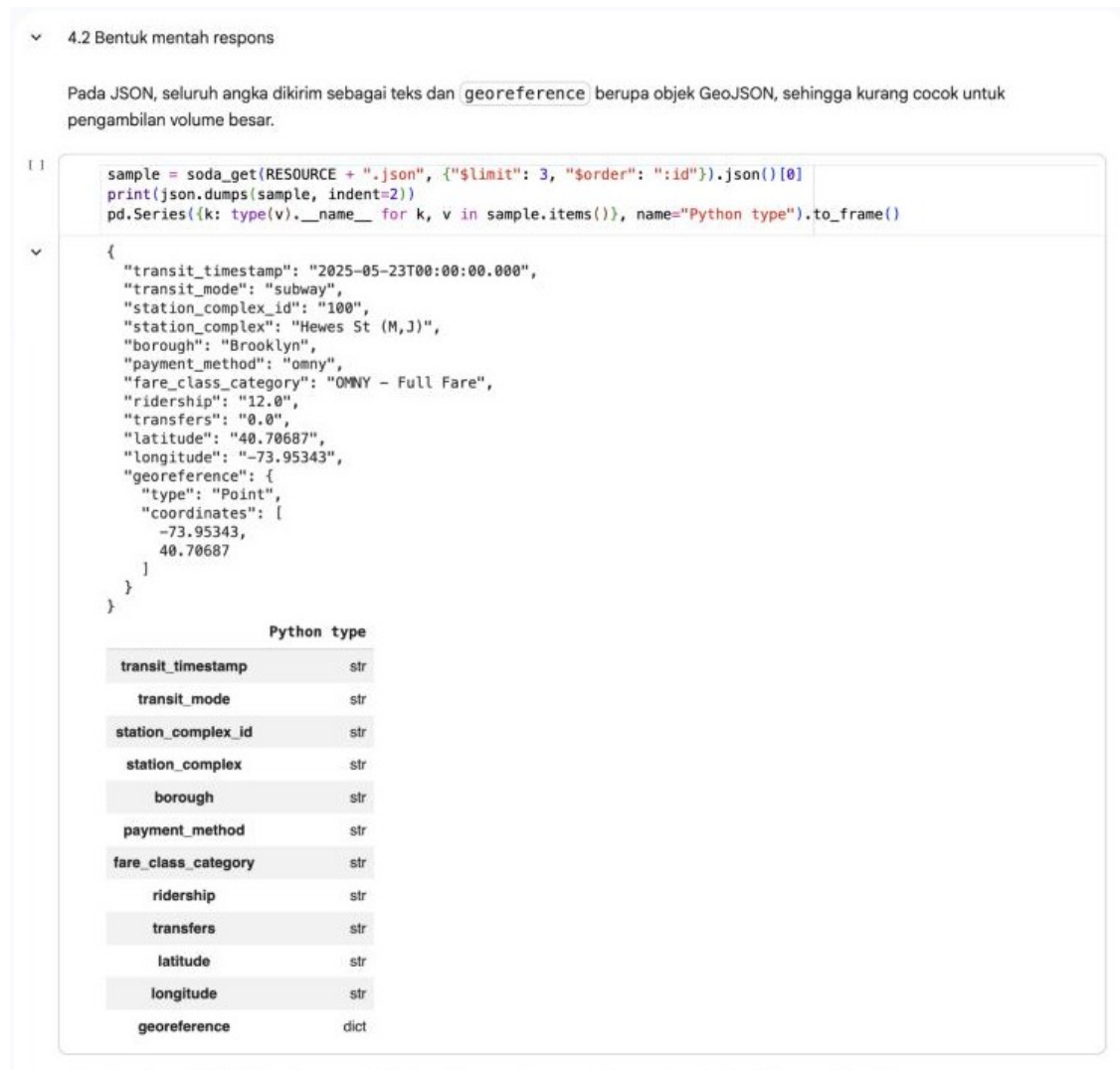
11 meta = soda_get(VIEW_META).json()
    info = meta["metadata"]["custom_fields"]["Dataset Summary"]

    pd.Series({
        "Name": meta["name"],
        "ID": meta["id"],
        "Owner": meta["attribution"],
        "Period": info["Time Period"],
        "Granularity": info["Granularity"],
        "Update frequency": info["Posting Frequency"],
        "Rows last updated (UTC)": dt.datetime.fromtimestamp(meta["rowsUpdatedAt"], dt.timezone.utc).strftime("%Y-%m-%d %H:%M"),
        "License": meta.get("license") or "(empty)",
    }, name="Value").to_frame()

```

	Value
Name	MTA Subway Hourly Ridership: Beginning 2025
ID	5wq4-mkjj
Owner	Metropolitan Transportation Authority
Period	Beginning 2025
Granularity	Hour, mode, subway station complex, fare medium, class of fare payment
Update frequency	Weekly
Rows last updated (UTC)	2026-09-23 14:46
License	(empty)

Gambar 1. Metadata dataset dari endpoint `/api/views`.



Gambar 2. Bentuk mentah respons JSON.

III. Karakteristik Awal Dataset

Tabel 3. Karakteristik awal dataset final

Karakteristik	Nilai
Dimensi	45.243.727 baris × 12 kolom
Rentang waktu	2025-01-01 00:00 sampai 2026-09-16 23:00 (14.975 jam, 624 hari)
Kompleks stasiun	428
Moda	3: <i>subway</i> (99,27% baris), <i>Staten Island Railway</i> (0,39%), <i>tram</i> (0,35%)
<i>Borough</i>	5: Brooklyn, Manhattan, Queens, Bronx, Staten Island
Metode pembayaran	2: OMNY (55,41% baris) dan MetroCard (44,59% baris)
Kelas tarif	12 kategori
<i>ridership</i>	min 1; median 7; rata-rata 49,49; maks 23.510; total 2.238.986.651
<i>transfers</i>	min -30; median 0; rata-rata 2,47; maks 2.556; total 111.767.861
Nilai kosong	0 pada seluruh kolom

Nilai-nilai di atas diambil dari pemeriksaan struktur awal pada notebook bagian 8 (Gambar 3–6). Temuan awal berikut hanya dicatat untuk tahap *assessing* dan *cleaning*; tidak ada perbaikan pada tahap ini:

- 226 baris `transfers` bernilai negatif.
- Satu jam tidak muncul, yaitu 2026-03-08 02:00 (bertepatan dengan pergantian ke waktu musim panas di New York).
- Satu kompleks stasiun (St George, ID 501) tercatat dengan dua moda.

8. Pemeriksaan Struktur Awal Dataset

Dataset final dibuka sebagai `view DuckDB (mta)` sehingga dapat dikueri dengan SQL tanpa memuat seluruh baris ke memori.

8.1 Dimensi dan tipe data

Jumlah baris, jumlah kolom, dan tipe data tiap kolom:

```
con.execute(f"CREATE OR REPLACE VIEW mta AS SELECT * FROM read_parquet('{FINAL_PATH}')
```

```
n_rows = con.sql("SELECT count(*) FROM mta").fetchone()[0]
print(f"Shape: {fmt(n_rows)} rows x {len(COLUMNS)} columns")
show_table(con.sql("DESCRIBE mta").df()[["column_name", "column_type"]].set_axis(["column", "type"], axis=1))
```

Shape: 45.243.727 rows x 12 columns

column	type
transit_timestamp	TIMESTAMP
transit_mode	VARCHAR
station_complex_id	VARCHAR
station_complex	VARCHAR
borough	VARCHAR
payment_method	VARCHAR
fare_class_category	VARCHAR
ridership	DOUBLE
transfers	DOUBLE
latitude	DOUBLE
longitude	DOUBLE
georeference	VARCHAR

Gambar 3. Dimensi dan tipe data dataset final.

8.3 Nilai kosong

Jumlah `NULL` pada semua kolom dan string kosong pada kolom teks.

```
TEXT_COLS = [f.name for f in SCHEMA if pa.types.is_string(f.type)]
q_null = ", ".join(f"count(*) - count({c}) AS {c}" for c in COLUMNS)
q_empty = ", ".join(f"count(*) FILTER (WHERE trim({c}) = '') AS {c}" for c in TEXT_COLS)

nulls = con.sql(f"SELECT {q_null} FROM mta").df().iloc[0]
empty = con.sql(f"SELECT {q_empty} FROM mta").df().iloc[0]
show_table(pd.DataFrame({"column": COLUMNS, "null_count": nulls.values, "empty_string_count": [empty.get(c, 0) for c in COLUMNS]}))
```

column	null_count	empty_string_count
transit_timestamp	0	0
transit_mode	0	0
station_complex_id	0	0
station_complex	0	0
borough	0	0
payment_method	0	0
fare_class_category	0	0
ridership	0	0
transfers	0	0
latitude	0	0
longitude	0	0
georeference	0	0

8.4 Kolom kategorik

Jumlah nilai unik tiap kolom teks:

```
unique = con.sql("SELECT " + ", ".join(f"count(DISTINCT {c}) AS {c}" for c in TEXT_COLS) + " FROM mta").df().T
show_table(unique.reset_index().set_axis(["column", "unique_values"], axis=1))
```

column	unique_values
transit_mode	3
station_complex_id	428
station_complex	428
borough	5
payment_method	2
fare_class_category	12
georeference	428

Gambar 4. Jumlah nilai kosong (0 pada seluruh kolom) dan jumlah nilai unik kolom teks.

8.5 Kolom numerik

Ringkasan statistik `ridership` dan `transfers`.

```

1 | def summarize(col):
    |     return (f"SELECT '{col}' AS variable, min({col}) AS min, quantile_cont({col}, 0.5) AS median, avg({col}) AS mean,
    |             f"quantile_cont({col}, 0.95) AS p95, max({col}) AS max, stddev({col}) AS std, sum({col}) AS total FROM mta")
    |
    | show_table(con.sql(" UNION ALL ").join(map(summarize, ["ridership", "transfers"]))).df(), 2)

```

variable	min	median	mean	p95	max	std	total
ridership	1,00	7,00	49,49	188,00	23.510,00	241,86	2.238.986.651,00
transfers	-30,00	0,00	2,47	9,00	2.556,00	17,74	111.767.861,00

8.6 Cakupan waktu

Rentang waktu serta jumlah jam dan hari unik:

```

1 | time_span = con.sql("""
    |     SELECT strftime(min(transit_timestamp), '%Y-%m-%d %H:%M') AS first_ts, strftime(max(transit_timestamp), '%Y-%m-%d %H:%M') AS last_ts,
    |     count(DISTINCT transit_timestamp) AS unique_hours, count(DISTINCT CAST(transit_timestamp AS DATE)) AS unique_days
    |     FROM mta""").df()
    | show_table(time_span)

```

first_ts	last_ts	unique_hours	unique_days
2025-01-01 00:00	2026-09-16 23:00	14.975	624

Gambar 5. Statistik kolom numerik dan cakupan waktu.

Jam yang tidak muncul pada rentang tersebut:

```

1 | missing_hours = con.sql("""
    |     SELECT h AS missing_hour
    |     FROM generate_series((SELECT min(transit_timestamp) FROM mta), (SELECT max(transit_timestamp) FROM mta), INTERVAL '1 hour') AS h
    |     WHERE h NOT IN (SELECT DISTINCT transit_timestamp FROM mta)""").df()
    | print(f"Missing hours: {len(missing_hours)}")
    | missing_hours

```

missing_hour
0 2026-03-08 02:00:00

8.7 Catatan awal untuk tahap berikutnya

Pemeriksaan cepat yang hanya mencatat temuan; tidak ada perbaikan pada tahap ini.

```

1 | show_table(con.sql("""
    |     SELECT count(*) FILTER (WHERE transfers < 0) AS negative_transfers,
    |     count(*) FILTER (WHERE transfers > ridership) AS transfers_exceeding_ridership,
    |     count(*) FILTER (WHERE ridership <= 0) AS nonpositive_ridership
    |     FROM mta""").df().T.reset_index().set_axis(["check", "row_count"], axis=1))

```

check	row_count
negative_transfers	226
transfers_exceeding_ridership	0
nonpositive_ridership	0

Kompleks stasiun yang tercatat dengan lebih dari satu moda:

```

1 | con.sql("""
    |     SELECT station_complex_id, station_complex, string_agg(DISTINCT transit_mode, ' ') AS modes
    |     FROM mta GROUP BY 1, 2 HAVING count(DISTINCT transit_mode) > 1""").df()

```

station_complex_id	station_complex	modes
0	501 St George (SIR)	subway, staten_island_railway

Gambar 6. Temuan awal yaitu jam yang tidak muncul, transfers negatif, dan stasiun dengan dua moda.

IV. Tahapan Gathering

Tabel 4. Tahapan *gathering*

No	Tahap	Keterangan	Bagian notebook
1	Persiapan lingkungan	Impor pustaka, parameter, dan fungsi akses API dengan <i>retry</i>	3
2	Verifikasi sumber	Metadata, bentuk mentah respons, dan skema 12 kolom	4
3	Fiksasi <i>snapshot</i>	<i>Cutoff</i> 2026-09-16 23:00; 45.243.727 baris, sama dengan nilai acuan	5.1
4	Rencana pengambilan	Jumlah baris per hari dari server untuk 624 hari	5.2
5	Uji satu hari	2025-01-01: 74.914 baris, sama dengan server	6.1
6	Pengambilan seluruh hari	624 hari, 6 unduhan paralel, selesai dalam 13,3 menit	6.2
7	Penggabungan	Harian → bulanan → satu file final	7
8	Pemeriksaan struktur awal	Dimensi, tipe, nilai kosong, sebaran, dan statistik	8
9	Validasi	Perbandingan dengan server, duplikat, dan urutan data	9

Jumlah baris di server sampai *cutoff* dicocokkan lebih dulu dengan nilai acuan *snapshot* (Gambar 7). Setelah itu jumlah baris setiap hari ditanyakan ke server sebagai rencana pengambilan (Gambar 8 dan 9).

Data diambil per hari lewat endpoint CSV dan dibaca dengan skema *pyarrow* yang tetap. Jumlah baris setiap hari dibandingkan dengan jumlah dari server; bila berbeda, pengambilan diulang hingga tiga kali. Hasilnya disimpan sebagai file Parquet harian melalui berkas sementara, sehingga file yang tersimpan selalu utuh. Pada run terakhir, 623 hari diunduh dalam 13,3 menit dan 1 hari dilewati karena sudah diunduh pada uji satu hari. Totalnya 45.243.727 baris, sama dengan server (Gambar 10).

5.1 Fiksasi *cutoff*

Dataset diperbarui setiap minggu, sehingga dua pengambilan pada waktu berbeda dapat menghasilkan data berbeda. *Snapshot* karena itu didefinisikan sebagai baris dengan `transit_timestamp <= CUTOFF_TS`, yaitu timestamp maksimum saat pengambilan pertama. Data setelahnya menjadi bahan tahap *streaming*. Sel berikut membandingkan status server dengan nilai acuan.

```
[ ] total = aggregate("count(*) AS n, min(transit_timestamp) AS first_ts, max(transit_timestamp) AS last_ts")[0]
snapshot = aggregate("count(*) AS n", where=f"transit_timestamp <= '{CUTOFF_TS}'")[0]
n_total, n_snapshot = int(total["n"]), int(snapshot["n"])

show_table(pd.DataFrame([
    ("Rows on server now", fmt(n_total)),
    ("Earliest timestamp", total["first_ts"][:19]),
    ("Latest timestamp", total["last_ts"][:19]),
    ("Fixed cutoff", CUTOFF_TS.replace("T", " ")),
    ("Snapshot rows (up to cutoff)", fmt(n_snapshot)),
    ("Rows after cutoff (not fetched)", fmt(n_total - n_snapshot)),
    ("Snapshot equals reference value?", "YES" if n_snapshot == EXPECTED["rows"] else "NO"),
], columns=["Check", "Result"]))
```

Check	Result
Rows on server now	45,243,727
Earliest timestamp	2025-01-01T00:00:00
Latest timestamp	2026-09-16T23:00:00
Fixed cutoff	2026-09-16 23:00:00
Snapshot rows (up to cutoff)	45,243,727
Rows after cutoff (not fetched)	0
Snapshot equals reference value?	YES

Gambar 7. Jumlah baris sampai *cutoff* sama dengan nilai acuan.

Data diambil per hari. Jumlah baris tiap hari ditanyakan ke server lebih dulu, sebagai rencana sekaligus acuan validasi.

```
per_day = aggregate("date_trunc_ymd(transit_timestamp) AS ymd, count(*) AS n", where=f"transit_timestamp <= '{CUT_DATE}'",
plan = {r["ymd"][:10]: int(r["n"]) for r in per_day if TEST_MONTHS is None or r["ymd"][:7] in TEST_MONTHS}
PLAN_ROWS = sum(plan.values())
print(f"{len(plan)} days, {fmt(PLAN_ROWS)} rows")
```

624 days, 45.243.727 rows

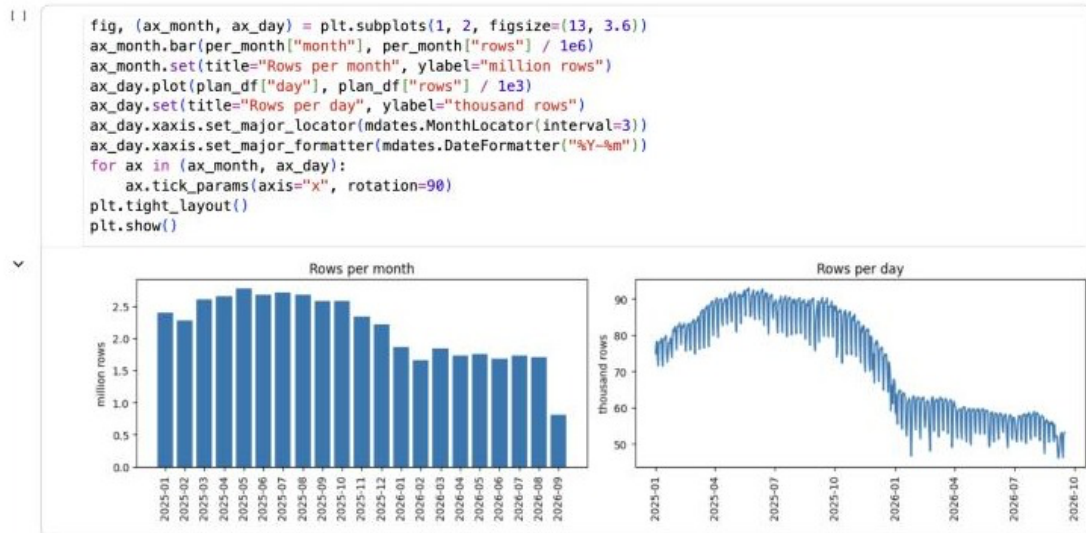
Rencana diringkas per bulan, lalu digambarkan:

```
plan_df = pd.DataFrame({"day": pd.to_datetime(list(plan)), "rows": list(plan.values())})
plan_df["month"] = plan_df["day"].dt.strftime("%Y-%m")
per_month = plan_df.groupby("month").agg(days=("day", "count"), rows=("rows", "sum")).reset_index()
show_table(per_month)
```

month	days	rows
2025-01	31	2.398.982
2025-02	28	2.273.357
2025-03	31	2.599.249
2025-04	30	2.646.409
2025-05	31	2.776.119
2025-06	30	2.673.221
2025-07	31	2.711.975
2025-08	31	2.681.448
2025-09	30	2.581.490
2025-10	31	2.578.816
2025-11	30	2.341.100
2025-12	31	2.209.615
2026-01	31	1.859.429
2026-02	28	1.658.281
2026-03	31	1.840.773
2026-04	30	1.725.407
2026-05	31	1.757.938
2026-06	30	1.683.047
2026-07	31	1.731.793
2026-08	31	1.706.084
2026-09	16	809.194

Gambar 8. Rencana pengambilan adalah jumlah hari dan baris per bulan.

Grafik jumlah baris per bulan dan per hari:



Gambar 9. Grafik jumlah baris per bulan dan per hari.

6.2 Pengambilan seluruh hari

Seluruh hari diunduh paralel dengan `WORKERS` thread.

```
seconds = {}
t0 = time.time()
with ThreadPoolExecutor(WORKERS) as pool:
    futures = {pool.submit(download_day, d, n): d for d, n in plan.items()}
    for f in tqdm(as_completed(futures), total=len(futures), desc="Downloading days", unit="day"):
        seconds[futures[f]] = f.result()
print(f"Finished in {(time.time() - t0) / 60, 1} minutes")
```

Downloading days: 100% |██████████| 624/624 [13:18<00:00, 1.28s/day]
Finished in 13,3 minutes

Ringkasan pengambilan. Jumlah baris dihitung dari metadata file harian; hari yang sudah ada dilewati dan tidak dihitung dalam waktu rata-rata.

```
n_daily = sum(pq.ParquetFile(day_file(d)).metadata.num_rows for d in plan)
downloaded = [s for s in seconds.values() if s > 0]

show_table(pd.DataFrame([
    ("Days collected", fmt(len(seconds))),
    ("Downloaded in this run", fmt(len(download))),
    ("Skipped (already present)", fmt(len(seconds) - len(download))),
    ("Rows in daily files", fmt(n_daily)),
    ("Rows on server", fmt(PLAN_ROWS)),
    ("Mean time per downloaded day (s)", fmt(sum(download) / len(download), 1) if download else "-"),
    ("Total size of daily files (MB)", fmt(sum(day_file(d).stat().st_size for d in plan) / 1e6, 1)),
], columns=["Summary", "Value"]))
```

Summary	Value
Days collected	624
Downloaded in this run	623
Skipped (already present)	1
Rows in daily files	45.243.727
Rows on server	45.243.727
Mean time per downloaded day (s)	7,7
Total size of daily files (MB)	374,3

Gambar 10. Pengambilan 624 hari secara paralel beserta ringkasannya.

V. Dataset dan File yang Digunakan

Tabel 5. File yang digunakan dan dihasilkan

Lapisan	Lokasi di Colab	Isi	Ukuran
Sumber	data.ny.gov/resource/5wq4-mkj	Data asli di server	–
Harian	/content/mta_subway/01_daily/YYYY-MM/YYYY-MM-DD.parquet	624 file	374,3 MB
Bulanan	/content/mta_subway/02_monthly/year=YYYY/month=MM/part-0.parquet	21 file	2,7–6,8 MB per file
Final	/content/mta_subway/03_final/mta_subway_hourly_ridership_2025-01_2026-09.parquet	1 file, 45.243.727 baris × 12 kolom	124,1 MB

Seluruh file berformat Apache Parquet dengan kompresi zstd.

VI. Proses Penggabungan Data

Seluruh potongan berasal dari satu dataset dan memiliki kolom yang sama, sehingga penggabungannya berupa *concat* vertikal (setara `UNION ALL`). *Join* tidak diperlukan karena tidak ada tabel lain yang perlu disambungkan. Operasi `pd.merge` dipakai pada tahap validasi untuk membandingkan agregat harian dataset final dengan server.

1. Tiga hari pertama digabung: $74.914 + 77.324 + 78.251 = 230.489$ baris (Gambar 11).
2. File harian setiap bulan dibaca dengan DuckDB, diurutkan menurut kunci (`transit_timestamp`, `station_complex_id`, `payment_method`, `fare_class_category`), lalu ditulis menjadi satu file per bulan. Jumlah baris tiap bulan sama dengan server dan rentang waktu antarbulan tidak tumpang tindih (Gambar 12).
3. Ke-21 file bulanan disambung berurutan menjadi satu file final berukuran 124,1 MB (Gambar 13).

DuckDB dipakai untuk data penuh karena membaca Parquet secara columnar tanpa memuat seluruh 45 juta baris ke RAM.



Gambar 11. Contoh penggabungan tiga hari pertama dengan `pd.concat`.

Ringkasan penggabungan per bulan; `rows` harus sama dengan `server_rows`.

```
manifest = pd.DataFrame([merge_month(m, n) for m, n in tqdm(zip(per_month["month"], per_month["rows"]), total=len(per_month["month"]))])
show_table(manifest.assign(first_ts=manifest["first_ts"].dt.strftime("%Y-%m-%d %H:%M"), last_ts=manifest["last_ts"].dt.strftime("%Y-%m-%d %H:%M")))
```

Merging months: 100% |██████████| 21/21 [02:05<00:00, 5.98s/month]

month	days	rows	server_rows	MB	first_ts	last_ts
2025-01	31	2.398.982	2.398.982	6,2	2025-01-01 00:00	2025-01-31 23:00
2025-02	28	2.273.357	2.273.357	5,7	2025-02-01 00:00	2025-02-28 23:00
2025-03	31	2.599.249	2.599.249	6,6	2025-03-01 00:00	2025-03-31 23:00
2025-04	30	2.646.409	2.646.409	6,4	2025-04-01 00:00	2025-04-30 23:00
2025-05	31	2.776.119	2.776.119	6,8	2025-05-01 00:00	2025-05-31 23:00
2025-06	30	2.673.221	2.673.221	6,6	2025-06-01 00:00	2025-06-30 23:00
2025-07	31	2.711.975	2.711.975	6,8	2025-07-01 00:00	2025-07-31 23:00
2025-08	31	2.681.448	2.681.448	6,8	2025-08-01 00:00	2025-08-31 23:00
2025-09	30	2.581.490	2.581.490	6,6	2025-09-01 00:00	2025-09-30 23:00
2025-10	31	2.578.816	2.578.816	6,7	2025-10-01 00:00	2025-10-31 23:00
2025-11	30	2.341.100	2.341.100	6,4	2025-11-01 00:00	2025-11-30 23:00
2025-12	31	2.209.615	2.209.615	6,3	2025-12-01 00:00	2025-12-31 23:00
2026-01	31	1.859.429	1.859.429	5,9	2026-01-01 00:00	2026-01-31 23:00
2026-02	28	1.658.281	1.658.281	5,2	2026-02-01 00:00	2026-02-28 23:00
2026-03	31	1.840.773	1.840.773	5,7	2026-03-01 00:00	2026-03-31 23:00
2026-04	30	1.725.407	1.725.407	5,4	2026-04-01 00:00	2026-04-30 23:00
2026-05	31	1.757.938	1.757.938	5,5	2026-05-01 00:00	2026-05-31 23:00
2026-06	30	1.683.047	1.683.047	5,3	2026-06-01 00:00	2026-06-30 23:00
2026-07	31	1.731.793	1.731.793	5,5	2026-07-01 00:00	2026-07-31 23:00
2026-08	31	1.706.084	1.706.084	5,4	2026-08-01 00:00	2026-08-31 23:00
2026-09	16	809.194	809.194	2,7	2026-09-01 00:00	2026-09-16 23:00

Pemeriksaan: jumlah baris tiap bulan sama dengan server, dan rentang waktu antarbulan tidak tumpang tindih, sehingga penyambungan berurutan tetap teratur menurut waktu.

```
assert (manifest["rows"] == manifest["server_rows"]).all(), "A month has a row count that differs from the server"
assert (manifest["first_ts"].iloc[1:].values > manifest["last_ts"].iloc[:-1].values).all(), "Monthly time ranges overlap"
print(f"{len(manifest)} monthly files, {fmt(manifest['rows'].sum())} rows: match the server and do not overlap.")
```

21 monthly files, 45.243.727 rows: match the server and do not overlap.

Gambar 12. Penggabungan harian ke bulanan

7.3 Bulanan ke satu file final

File bulanan disambung berurutan menjadi satu file. `hive_partitioning=false` mencegah DuckDB menambahkan kolom `year` dan `month` dari nama folder.

```
FINAL_PATH = FINAL_DIR / f"mta_subway_hourly_ridership_{min(plan)[:7]}_{max(plan)[:7]}.parquet"
file_list = ", ".join(f"{month_file(m)}" for m in per_month["month"])

con.execute(f"COPY (SELECT * FROM read_parquet({file_list}), hive_partitioning=false) TO '{FINAL_PATH}' ((COPY, print(f'Final file : {FINAL_PATH.name}')) print(f'Size : {fmt(FINAL_PATH.stat().st_size / 1e6, 1)} MB"))")
```

Final file : mta_subway_hourly_ridership_2025-01_2026-09.parquet
Size : 124,1 MB

Gambar 13. Penggabungan bulanan ke satu file final.

1.7 Kendala dan Cara Mengatasinya

Tabel 6. Kendala dan cara mengatasinya

No	Kendala	Cara mengatasi
1	Data besar (45,2 juta baris), tidak praktis diambil sekaligus dan berisiko membebani RAM	Data diambil per hari (624 potongan), disimpan sebagai Parquet terkompresi, lalu digabung dengan DuckDB
2	Respons JSON mengirim angka sebagai teks dan <i>georeference</i> sebagai objek GeoJSON	Memakai endpoint CSV dengan skema kolom <i>pyarrow</i> yang ditetapkan di awal
3	API publik dapat gagal sementara (<i>timeout</i> , kode 429 atau 5xx)	Fungsi <i>soda_get</i> mengulang permintaan hingga 8 kali dengan jeda yang makin panjang; jumlah baris tiap hari juga divalidasi dan diulang hingga 3 kali bila berbeda
4	Dataset diperbarui setiap Rabu. Selain menambah minggu terbaru, pembaruan juga menambah baris terlambat untuk tanggal lama dan merevisi data berbulan-bulan ke belakang (pembaruan 23 September mengubah 6,5 juta baris).	Snapshot difiksasi dengan cutoff dan nilai acuan, divalidasi terhadap server saat pengambilan, lalu disimpan sebagai file Parquet final yang menjadi dataset terfiksasi.
5	Sesi Colab bersifat sementara; isi folder <i>/content</i> hilang saat sesi berakhir	Hari yang filenya sudah ada dilewati, sehingga proses dapat dilanjutkan; menyediakan opsi <i>SAVE_TO_DRIVE</i> untuk menyalin dataset final ke Google Drive

VII. Hasil Akhir Gathering

Proses *gathering* menghasilkan satu dataset final yang lengkap dan sama dengan sumbernya. Agregat harian (jumlah baris, total *ridership*, total *transfers*) untuk 624 hari sama dengan server (Gambar 14), tidak ada duplikat kunci dan data terurut (Gambar 15), serta tidak ada nilai kosong.

Tabel 7. Ringkasan dataset final

Atribut	Nilai
Nama file	<i>mta_subway_hourly_ridership_2025-01_2026-09.parquet</i>
Format	Apache Parquet (kompresi <i>zstd</i>)
Ukuran	124,1 MB
Baris × kolom	$45.243.727 \times 12$
Rentang waktu	2025-01-01 00:00 sampai 2026-09-16 23:00
Kompleks stasiun	428
Total <i>ridership</i>	2.238.986.651
Waktu proses di Colab	Pengambilan 13,3 menit; penggabungan bulanan sekitar 2 menit
Validasi	10 dari 10 pemeriksaan PASS

9.1 Perbandingan per hari dengan server

Server menghitung agregat harian dari data aslinya (jumlah baris, total `ridership`, total `transfers`). Hasilnya digabung (`pd.merge`) dengan agregat yang dihitung dari dataset final, berdasarkan tanggal.

```
11 server = pd.DataFrame(aggregate(
    "date_trunc_ymd(transit_timestamp) AS ymd, count(*) AS n, sum(ridership) AS r, sum(transfers) AS t",
    where=f"transit_timestamp <= '{CUTOFF_TS}'", group="ymd"))
server["ymd"] = server["ymd"].str[:10]
server = server[server["ymd"].isin(plan)].astype({"n": int, "r": float, "t": float})
```

Agregat yang sama dihitung dari dataset final:

```
11 local = con.sql("""
    SELECT strftime(CAST(transit_timestamp AS DATE), '%Y-%m-%d') AS ymd, count(*) AS n, sum(ridership) AS r, sum
    FROM mta GROUP BY 1""").df()
```

Kedua agregat dibandingkan per hari dengan `pd.merge`:

```
11 compare = server.merge(local, on="ymd", how="outer", suffixes=("_server", "_final"), indicator=True)
mismatch = compare[["n_server", "r_server", "t_server"]].values != compare[["n_final", "r_final", "t_final"]].va
n_diff = int(((compare["_merge"] != "both") | mismatch.any(axis=1)).sum())

print(f"Days compared: {len(compare)} | days that differ: {n_diff}")
show_table(compare.head(5).drop(columns="_merge"))
```

```
Days compared: 624 | days that differ: 0
   ymd  n_server  r_server  t_server  n_final  r_final  t_final
2025-01-01    74.914    1.813.725    54.931    74.914    1.813.725    54.931
2025-01-02    77.324    3.433.271    160.717    77.324    3.433.271    160.717
2025-01-03    78.251    3.467.018    161.389    78.251    3.467.018    161.389
2025-01-04    73.385    2.344.771    86.910    73.385    2.344.771    86.910
2025-01-05    71.505    1.818.179    65.133    71.505    1.818.179    65.133
```

Gambar 14. Perbandingan agregat harian dengan server.

9.2 Keunikan dan urutan baris

Data sudah terurut menurut `SORT_KEYS`, sehingga duplikat pasti berdampingan. Membandingkan tiap baris dengan baris sebelumnya (`lag`) cukup untuk memeriksa duplikat dan urutan, dan lebih hemat memori daripada mengelompokkan 45 juta kunci. Pemeriksaan dilakukan pada tiap file bulanan (kunci lengkap) dan pada file final (`transit_timestamp` tidak boleh mundur).

```
11 def check_order(path):
    keys = ", ".join(SORT_KEYS)
    dups, unsorted = con.execute(f"""
        WITH t AS (
            SELECT ({keys}) AS k, lag(({keys})) OVER (ORDER BY file_row_number) AS prev
            FROM read_parquet('{path}', file_row_number = true)
            SELECT count(*) FILTER (WHERE k = prev), count(*) FILTER (WHERE k < prev) FROM t""").fetchone()
    return {"dups": dups, "unsorted": unsorted}
```

Pemeriksaan pada tiap file bulanan:

```
11 order_check = pd.DataFrame([{"month": m, **check_order(month_file(m))} for m in tqdm(per_month["month"], desc="Cl
print(f"Duplicate keys (all monthly files): {order_check['dups'].sum()}")
print(f"Unsorted rows (monthly files) : {order_check['unsorted'].sum()}")
```

```
Checking order and duplicates: 100%| 21/21 [00:32<00:00, 1.55s/mont
Duplicate keys (all monthly files): 0
Unsorted rows (monthly files) : 0
```

Pemeriksaan pada file final:

```
11 final_backward = con.execute(f"""
    WITH t AS (
        SELECT transit_timestamp AS ts, lag(transit_timestamp) OVER (ORDER BY file_row_number) AS prev
        FROM read_parquet('{FINAL_PATH}', file_row_number = true)
        SELECT count(*) FILTER (WHERE ts < prev) FROM t""").fetchone()[0]
print(f"Timestamps going backward (final) : {final_backward}")
```

```
Timestamps going backward (final) : 0
```

Gambar 15. Pemeriksaan duplikat kunci dan urutan data

10. Dataset Akhir Hasil *Gathering*

Ringkasan dataset akhir yang diserahkan ke tahap *assessing* dan *cleaning*.

```
show_table(pd.DataFrame([
    ("File name", FINAL_PATH.name),
    ("Folder", str(FINAL_DIR)),
    ("Format", "Apache Parquet (zstd compression)"),
    ("File size", f"{fmt(FINAL_PATH.stat().st_size / 1e6, 1)} MB"),
    ("Rows", fmt(n_rows)),
    ("Columns", len(COLUMNS)),
    ("Time range", f"{time_span.loc[0, 'first_ts']} to {time_span.loc[0, 'last_ts']}"),
    ("Station complexes", fmt(con.sql("SELECT count(DISTINCT station_complex_id) FROM mta").fetchone()[0])),
    ("Total ridership", fmt(round(r_total))),
    ("Snapshot cutoff", CUTOFF_TS.replace("T", " ")),
], columns=["Attribute", "Value"]))
```

Attribute	Value
File name	mta_subway_hourly_ridership_2025-01_2026-09.parquet
Folder	/content/mta_subway/03_final
Format	Apache Parquet (zstd compression)
File size	124,1 MB
Rows	45.243.727
Columns	12
Time range	2025-01-01 00:00 to 2026-09-16 23:00
Station complexes	428
Total ridership	2.238.986.651
Snapshot cutoff	2026-09-16 23:00:00

Gambar 16. Ringkasan dataset akhir hasil *gathering*.